

COGNOME:

NOME:

MATRICOLA:

Appello di Algoritmi e Strutture Dati (9CFU), 18 Settembre 2025

Parte I - Usare voto prova parziale - SI() NO()

Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

- (A) 2025^{2026} , $\log n^{2025}$, $\frac{n^2}{(\log n)^{2025}}$, $\sqrt{n}$, $2^{\sqrt{n} \log n}$
- (B) $\sqrt{n}$, $\sum_{i=0}^{\log n} 2^i$, $2^{\sqrt{n}}$, $n^{(\log n)^2}$, 2^n
- (C) $\log n$, $\sum_{i=1}^n \log i$, $n \log n$, $\log n!$, $n^{1.001}$
- (D) Nessuna delle altre

Esercizio 2 [4 punti]

Si consideri il seguente algoritmo, in cui l'input è un intero positivo n , e se ne determini il tempo di esecuzione nel caso peggiore. Si assuma che ciascuna somma sia eseguibile in $O(1)$.

Algorithm ALGO(n)

```

1: if  $n = 1$  or  $n = 2$  then
2:    $x = x + 1$ 
3: if  $n > 2$  then
4:   ALGO( $n/4$ )
5:   ALGO( $n/4$ )
6: if  $n > 2025$  then
7:   for  $i = 1, 2, 3, \dots, \sqrt{n}$  do
8:      $x = x + 1$ 

```

$$T(n) = 2T(n/4) + O(\sqrt{n})$$

$$a=2 \quad b=4 \quad n^{\log_4 2} = n^{1/2} = \sqrt{n} = f(n)$$

$$T(n) = \Theta(\sqrt{n} \log n)$$

- (A) $\Theta(n \log n)$
- (B) $\Theta(n)$
- (C) $\Theta(\sqrt{n})$
- ~~(D)~~ Nessuna delle altre

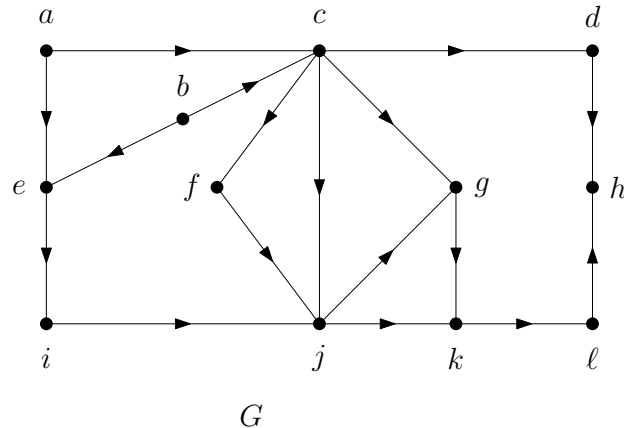
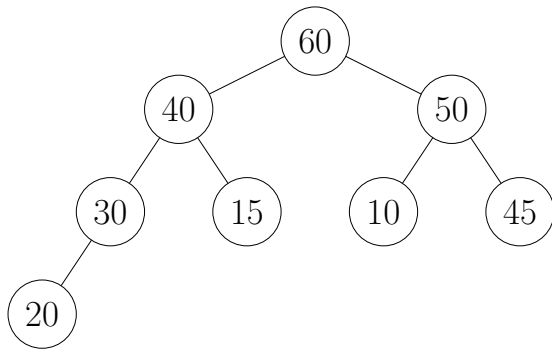
Esercizio 3 [3 punti]

Quale delle seguenti affermazioni è vera?

- (A) Dato un array A di dimensione n , l'algoritmo ALGO1 invoca l'algoritmo ALGO2 per ogni elemento $A[i]$, e ALGO2 richiede un tempo di esecuzione $O(i)$ quando viene invocato con $A[i]$. Allora il tempo di esecuzione di ALGO1 è $\Theta(n)$. → $\text{for } i \rightarrow n \quad 1+2+3+\dots+n = \Theta(n^2)$
- ~~(B)~~ INSERTION-SORT è un algoritmo di ordinamento stabile.
- (C) Un albero di decisione per l'ordinamento di n elementi ha altezza $\Omega(n^2)$.
- (D) Nessuna delle altre. → $n!$ possibili permutazioni → $h = \log n! = n \log n$

Esercizio 4 [3 punti]

Si consideri l'albero in figura, che rappresenta un max-heap A . La rappresentazione di A tramite array dopo chiamate, nell'ordine, a $\text{MAX-HEAP-INSERT}(A, 53)$, $\text{HEAP-EXTRACT-MAX}(A)$, $\text{MAX-HEAP-INSERT}(A, 35)$, e $\text{HEAP-EXTRACT-MAX}(A)$ è: (A) $\langle 50, 45, 40, 35, 15, 10, 30, 20 \rangle$ (B) $\langle 50, 40, 45, 30, 15, 10, 35, 20 \rangle$ (C) $\langle 50, 40, 45, 35, 20, 10, 30, 15 \rangle$ ~~(D) Nessuna delle altre~~



Esercizio 5 [4 punti]

Si consideri il grafo diretto G in figura. Si effettui una visita in profondità di G a partire da a in cui i vertici sono ordinati alfabeticamente, così come ciascuna lista di adiacenza. Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.
- (B) L'insieme degli archi in avanti è $\{(c, j), (c, g), (j, k), (\ell, h)\}$.
- (C) G non ammette alcun ordinamento topologico.
- ~~(D) I vertici c e g appartengono alla stessa componente fortemente connessa.~~

Esercizio 6 [3 punti]

Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.
- (B) È possibile rimuovere l'elemento di indice i da un max-heap A mantenendo la proprietà del max-heap nel seguente modo: scambio $A[i]$ con $A[A.\text{heap-size}]$, decremento $A.\text{heap-size}$ di 1 e infine chiamo $\text{MAX-HEAPIFY}(A, i)$.
- (C) Una visita in-order di un max-heap permette di ottenere un ordinamento crescente.
- ~~(D) Una visita in-order di un albero binario di ricerca permette di ottenere un ordinamento crescente.~~

SBARRAMENTO - 12 punti sugli esercizi precedenti

Esercizio 7 [8 punti] Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

Esercizio 8 [4 punti] Siano $G = (V, E)$ un grafo diretto e $s, t \in V$. Assegniamo a ciascun arco $e \in E$ un valore reale $r(e)$ (la sua *affidabilità*), tale che $0 < r(e) \leq 1$. L'affidabilità di un cammino P in G è definita come il prodotto delle affidabilità degli archi di P . Vogliamo trovare un cammino da s a t di affidabilità massima. Dimostrare che questo problema può essere ridotto ad un problema di cammini minimi in un grafo pesato ausiliario.

Sugg.: Considerare le proprietà del logaritmo.

Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) 2025^{2026} , $\log n^{2025}$, $\frac{n^2}{(\log n)^{2025}}$, $\sqrt{n}$, $2^{\sqrt{n} \log n}$

(B) $\sqrt{n}$, $\sum_{i=0}^{\log n} 2^i$, $2^{\sqrt{n}}$, $n^{(\log n)^2}$, 2^n

(C) $\log n$, $\sum_{i=1}^n \log i$, $n \log n$, $\log n!$, $n^{1.001}$

(D) Nessuna delle altre

A. $2025^{2026} \leq \sum_{i=0}^{\log n} 2^i \leq \frac{n^2}{(\log n)^{2025}} > \sqrt{n}$

• $\sum_{i=0}^{\log n} 2^i \approx 2^{\log n + 1} = 2^{\log n} \cdot 2 = 2n$

• $\frac{n^2}{(\log n)^{2025}} = n^2 \cdot \frac{1}{(\log n)^{2025}}$

• $\sqrt{n} = n^{1/2}$

B. $\sqrt{n} \leq \sum_{i=0}^{\log n} 2^i \leq 2^{\sqrt{n}} \leq n^{(\log n)^2} \leq 2^n$

• $\sqrt{n} = n^{1/2}$

• $\sum_{i=0}^{\log n} 2^i \approx 2^{\log n + 1} = 2^{\log n} \cdot 2^1 = 2n$

• $2^{\sqrt{n}} = 2^{n^{1/2}}$

• $n^{(\log n)^2}$

• 2^n

applico il log. : $\log 2^{\sqrt{n}}$ $\log n^{(\log n)^2}$ $\log 2^n$ $\log n^{(\log n)^2}$
 $\sqrt{n} \log 2$ $(\log n)^2 \log n$ $n \log 2$ $(\log n)^2 \log n$
 $\sqrt{n} < (\log n)^3$ $n < (\log n)^3$

C. $\log n \leq \sum_{i=1}^n \log i \leq n \log n \leq \log n! \leq n^{1.001}$

• $\sum_{i=1}^n \log i = \log n! = n \log n$

• $n^{1.001} = n^{1+0.001}$

Esercizio 7 [8 punti] Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

Dijkstra: è un algoritmo greedy utilizzato per risolvere il problema dei Cammini minimi da sorgente singola su un grafo pesato e orientato (o non orientato), con peso positivo (>0)

- Il funzionamento:
1. Viene mantenuta una coda di min-priorità Q contenente tutti i vertici del grafo, dove la chiave di ciascun vertice è la sua stima corrente del cammino minimo $C.V.d$
 2. Ad ogni passo, l'algoritmo estrae da Q il vertice u con la stima di distanza minima assoluta (greedy) e lo inserisce nell'insieme dei nodi
 3. L'algoritmo esegue il rilassamento di tutti gli archi uscenti da u , Per ogni vicino v , controlla se il cammino passante per u è più conveniente della stima attuale $v.d$. Se lo è, aggiorna $v.d$ e il predecessore $v.p$

Dijkstra(G, w, s)

INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(|V|)$

$S = \emptyset$

$Q = G.V$

Mentre $Q \neq \emptyset$ do

$u = \text{EXTRACT-MIN}(Q)$ $\rightarrow O(|V| \log |V|)$

$S = S \cup \{u\}$

Per ogni vertice $v \in G.Adj[u]$ do

RELAX(u, v, w)

DECREASE-KEY($Q, v, v.d$) $\rightarrow O(|E| \log |V|)$, $|E| \geq |V| - 1 \rightarrow$ Graf connesso

$\text{tot} = O(|E| \log |V| + |V| \log |V|) = O(|E| \log |V|)$